Ponte Servidor Caseiro e AWS

Backup diário e failover do SOUFER Tools

Projeto: SOUFER Tools (PI 2026.2)

Status: proposta para decisão da equipe (nada disto foi implantado ainda)

Relacionados: `dokploy.md` (servidor caseiro), `infra-plano.md` (plano de nuvem), `custos-nuvem.md` (custos), guia "Como enviar o projeto para a AWS" (EC2 atual, fora do repositório)

1. Resumo da proposta

Pergunta	Resposta
O que é o sistema "principal"?	O **servidor caseiro** (Dokploy + Docker + Traefik + PostgreSQL em contêiner).
Qual o papel da AWS?	Guardar **cópia diária** do que está no servidor caseiro e **assumir o serviço** se ele cair.
Dá para fazer tudo dentro da EC2?	**Dá**, e é o caminho mais barato e o que a equipe já domina. Mas a EC2 sozinha deixa buracos: o backup mora no mesmo disco, não há alarme independente e o DNS não troca sozinho.
Quais serviços AWS a mais valem a pena?	**S3** (cofre dos backups) e **SNS/CloudWatch** (alerta). **Route 53** só se o Learner Lab liberar; senão, DNS gratuito (DuckDNS) ou domínio próprio com API de atualização. Detalhes na seção 4.
Como a EC2 fala com a casa?	Por um **túnel WireGuard** iniciado pela casa (funciona atrás de roteador/CGNAT, sem abrir porta no roteador).
Quem puxa o backup?	A **EC2 puxa** do servidor caseiro. Assim a casa não precisa guardar chave da AWS (no Learner Lab as credenciais expiram e não dá para criar usuário IAM).
Quanto custa?	Standby "frio" (EC2 desligada, só liga se a casa cair): **~US$ 5,75/mês**. Standby "quente" (EC2 sempre ligada): **~US$ 36/mês**. Ver seção 9.
Maior risco	O **AWS Academy Learner Lab desliga a sessão em ~4 h** (informado no guia de deploy). Para a apresentação serve como demonstração; para uso real na Soufer precisa de conta AWS própria. Ver seção 2.

Recomendação: começar pela Fase 1 (túnel + backup diário para o S3 + restauração testada + roteiro manual de troca) e só depois automatizar a troca (Fase 2). A replicação contínua do banco (Fase 3) fica como "se sobrar tempo".

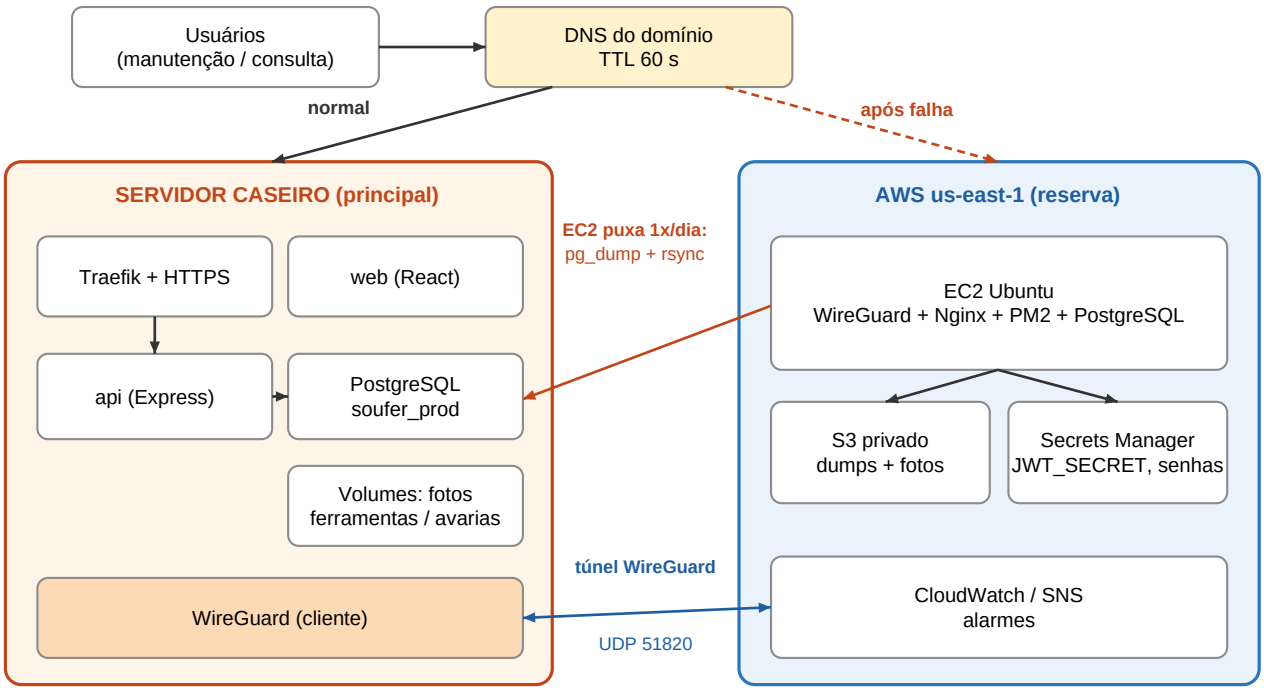
2. Premissas e limites

1. **Servidor caseiro:** Dokploy com três serviços (`soufer-tools-web`, `soufer-tools-api`, `soufer-postgres`), HTTPS pelo Traefik, conforme `dokploy.md`. A porta 3000 da API **não** é publicada (exigência do `TRUST_PROXY_HOPS=1`).
2. **EC2 atual** (guia de deploy, Partes 1 a 9): Ubuntu 24.04, `t3.medium`, Nginx + API Node (PM2) + PostgreSQL 16 + front estático, Elastic IP `32.202.0.79`, segredos no Secrets Manager (`soufer-tools/prod`) e `LabRole`.
3. **Learner Lab:** a sessão dura cerca de 4 horas, só existe a `LabRole`, e as credenciais temporárias rotacionam. Consequências:

- failover **24 h por dia** não é garantido enquanto estivermos no Lab (a EC2 para quando o Lab é encerrado);
- não há como criar um usuário IAM com chave fixa para o servidor caseiro enviar arquivos ao S3;
- serviços como Route 53, SNS, EventBridge e Lambda **podem** estar bloqueados: testar antes de depender deles.

4. Em conta AWS real da Soufer, nada disso vale: dá para usar usuário IAM com permissão mínima, Route 53 e Lambda.
5. **Alinhamento com o PDF do PI:** o PDF exige entrega digital, código e documentação no GitHub e demonstração funcional. Esta proposta não conflita com ele; o que precisa estar de pé na apresentação (23 a 27/11) é a aplicação acessível e, de preferência, uma demonstração do backup e da troca.

3. Arquitetura proposta



Fluxo normal: o usuário abre `https://app.<dominio>`; o DNS aponta para a casa; a EC2 só puxa o backup. **Fluxo de falha:** o vigia detecta que a casa não responde, a EC2 restaura/ativa o banco e a API, o DNS passa a apontar para o Elastic IP da EC2.

4. "Dá para fazer tudo na EC2?" — o que fica onde

Função	Só na EC2	Com serviço AWS	Recomendação
Rodar API, front e banco de reserva	Sim (já está pronto)	RDS/S3+CloudFront custam mais e mudam a stack	**EC2**
Agendar o backup diário	`systemd timer` ou cron	EventBridge	**EC2** (mais simples)
Guardar os backups	Disco da própria EC2 (se a EC2 morrer, perde junto)	**S3** (US$ 0,023/GB-mês, versionamento, criptografia)	**S3** (cópia fora do disco)

Função	Só na EC2	Com serviço AWS	Recomendação
Segredos (`JWT_SECRET`, senhas)	`.env` com `chmod 600`	**Secrets Manager** (já em uso)	**Secrets Manager**
Detectar que a casa caiu	Script na EC2 (quem vigia e quem assume é o mesmo; se a EC2 cair, ninguém avisa)	CloudWatch Alarm / Route 53 Health Check / monitor externo	Script na EC2 **+** um monitor externo gratuito (ex.: UptimeRobot) para alertar a equipe
Avisar a equipe	e-mail via script	**SNS**	SNS se liberado; senão webhook do Discord/WhatsApp pelo script
Trocar o DNS	API do provedor de DNS chamada pelo script	**Route 53** (failover automático com health check)	Route 53 se liberado; senão DuckDNS/domínio próprio + script
Ligar a EC2 só na falha (standby frio)	Impossível (ela mesma está desligada)	EventBridge + Lambda, ou o script rodar em outra máquina	Só em conta real. No Lab, usar standby "quente" ou ligar à mão

Conclusão: **EC2 + S3 + Secrets Manager** é o conjunto mínimo e honesto. O resto é opcional e depende do que o Lab libera.

5. Conectividade: o túnel

- **Tecnologia:** WireGuard (leve, uma chave por ponta, UDP 51820). Alternativa mais fácil de montar: Tailscale (gratuito para uso pessoal), com a mesma ideia.
- **Quem conecta:** o servidor caseiro **disca para a EC2** (o Elastic IP é fixo; a casa tem IP dinâmico/CGNAT). Usar `PersistentKeepalive = 25` no lado da casa.
- **Rede do túnel:** `10.8.0.0/24` (EC2 = `10.8.0.1`, casa = `10.8.0.2`).
- **Security Group da EC2:** abrir **UDP 51820** (origem qualquer, pois o IP da casa muda). SSH continua como está.
- **Firewall da casa:** só aceitar do túnel (`10.8.0.1`) a porta do PostgreSQL e o SSH usado no rsync.
- **PostgreSQL da casa:** o serviço `soufer-postgres` do Dokploy precisa de porta externa publicada **apenas no IP do túnel** (ex.: `10.8.0.2:5432`), nunca em `0.0.0.0`. Criar um papel **somente leitura** para o backup:

```
CREATE ROLE backup_ro LOGIN PASSWORD '<senha forte, só letras e números>';
GRANT CONNECT ON DATABASE soufer_prod TO backup_ro;
GRANT USAGE ON SCHEMA public TO backup_ro;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO backup_ro;
GRANT SELECT ON ALL SEQUENCES IN SCHEMA public TO backup_ro;
ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO backup_ro;
```

As migrations criam tabelas novas com o dono `postgres`/`soufer`; o `ALTER DEFAULT PRIVILEGES` precisa ser executado **pelo papel que roda as migrations**, senão tabelas novas ficam fora do backup. Validar com `pg_dump` após a próxima migration.

- **Versão do `pg_dump`:** o cliente da EC2 precisa ser **igual ou mais novo** que o PostgreSQL da casa. Conferir a imagem usada no Dokploy (`SELECT version();`) antes de fechar a versão na EC2.

6. Backup diário (a EC2 puxa)

6.1 O que entra no backup

Item	De onde vem	Como	Observação
Banco `soufer_prod`	PostgreSQL da casa	`pg_dump -Fc` pelo túnel	Formato custom, restaura com `pg_restore`. Contém hash bcrypt e dados de colaboradores: tratar como dado sensível (LGPD).
Fotos de ferramentas	`UPLOADS_DIR` (`/app/data/ferramentas`)	`rsync` pelo túnel	Volume Docker do Dokploy: descobrir o caminho real no host.
Fotos de avaria/perda	`AVARIAS_DIR` (`/app/data/avarias`)	`rsync` pelo túnel	Idem.
Variáveis da API	Dokploy → Environment	Cópia manual **uma vez**, no Secrets Manager e no `.env` da EC2	Não vai para o S3 em texto puro.
Versão do código	Branch/commit em produção na casa	Registrar no nome do backup (`git rev-parse` ou tag)	A EC2 precisa subir a **mesma versão** do dump (as migrations conferem o esquema).

6.2 Rotina

1. `systemd timer` na EC2, todo dia às **03:30** (fora do horário da manutenção).
2. Puxa o dump e valida com `pg_restore -l` (arquivo ilegível = falha, alerta).
3. Sincroniza as pastas de fotos.
4. Envia tudo para o **S3** (`aws s3 cp` com criptografia `AES256`), em `s3://soufer-tools-backup-<sufixo>/AAAA-MM-DD/`.
5. Mantém **14 dias** no disco da EC2 e **30 dias** no S3 (regra de ciclo de vida); o mais recente fica sempre à mão.
6. Registra sucesso/falha em log e avisa em caso de falha.

6.3 Restauração (testar antes de confiar)

```
sudo -u postgres createdb -O soufer soufer_prod
pg_restore -h 127.0.0.1 -U soufer -d soufer_prod --no-owner --clean --if-exists soufer_AAAA-MM-DD.dump
cd ~/PI-2026.2/api && scripts/com-segredos.sh npm run db:migrate   # idempotente: só aplica o que faltar
```

Teste de restauração obrigatório: restaurar um dump real na EC2 e conferir a contagem de ferramentas, empréstimos e colaboradores contra a casa. Backup nunca restaurado não conta como backup.

6.4 Alternativa: backup nativo do Dokploy

O Dokploy agenda backup do banco para um destino S3. Funcionaria, **mas** exige chave de acesso fixa da AWS, e o Learner Lab não oferece isso. Em conta real é a opção mais simples; no Lab, usar o modelo "EC2 puxa".

7. Failover: como a EC2 assume

7.1 Detecção

- Vigia na EC2 (`systemd timer` a cada 30 s) consulta `https://app.<dominio>/v1/health` pela internet pública (o mesmo caminho do usuário) e confere `"status":"ok"`.
- **Gatilho:** 6 falhas seguidas (≈ 3 minutos) **e** nenhum sinal de vida da casa no túnel (handshake do WireGuard parado). Se só o túnel caiu e o site público responde, **não troca**: só alerta.
- Um **monitor externo** (UptimeRobot ou CloudWatch Synthetics) avisa a equipe, porque se a própria EC2 estiver fora ninguém sabe.

7.2 Promoção (o que a EC2 faz)

1. Alerta "iniciando assunção".
2. Restaura o último dump (se o standby não estiver com banco já carregado) e roda `db:migrate`.
3. Sobe a API com `scripts/com-segredos.sh` (mesmos `JWT_SECRET` e `DB_PASSWORD` do Secrets Manager).
4. Confere `curl http://127.0.0.1:3000/v1/health` e `curl -I https://<dominio-da-ec2>/`.
5. Atualiza o DNS para o Elastic IP da EC2.
6. Grava a marca `/var/lib/soufer/MODO_RESERVA_ATIVO` e avisa a equipe com a hora do último backup (o que pode ter se perdido).

7.3 Pontos que quebram a troca se ninguém pensar neles antes

Ponto	Problema	Solução
Domínio `sslip.io`	O nome embute o IP (`32-202-0-79.sslip.io`); não existe como apontar o mesmo nome para dois IPs.	Usar **domínio de verdade** com TTL 60 s (`.com.br` no Registro.br, ou subdomínio DuckDNS). Já citado no guia como plano B (item 8.6).
Certificado HTTPS na EC2	O Let's Encrypt por HTTP-01 só emite se o DNS já apontar para a EC2, e no momento da falha é tarde.	Emitir o certificado da EC2 **antes**, por **DNS-01** (plugin do provedor de DNS), ou manter um nome próprio da reserva (`reserva.<dominio>`) já com certificado.
`JWT_SECRET` **diferente**	Todo mundo seria deslogado (sessões de 7 dias).	Usar o **mesmo** `JWT_SECRET` nos dois lados (copiar uma vez para o Secrets Manager).
`CORS_ORIGIN`	Front na EC2 chamando outra origem → erro de CORS.	Mesmo domínio nos dois lados (o DNS é que muda), então `CORS_ORIGIN` não muda.
`TRUST_PROXY_HOPS`	Sem `1`, o limite de tentativas de login passa a valer para todos juntos.	Nginx da EC2 já exige `TRUST_PROXY_HOPS=1`; manter.
Versões diferentes	Casa em `develop`, EC2 em `main`: o dump pode ter migration que a EC2 não conhece.	Fixar a mesma tag/commit nas duas pontas e registrar a versão no backup.
Seed	Rodar `db:seed` na reserva cria contas `123456`.	Nunca rodar; a API já recusa com `NODE_ENV=production`.
Notificações e feriados	Os jobs da API (BrasilAPI, notificações) rodam nos dois lados e duplicam.	Manter a API da reserva **desligada** até a promoção.

8. Volta para a casa (failback)

A volta **nunca é automática**: se a casa reaparecer sozinha com o banco de ontem, ela sobrescreveria o que a EC2 gravou (dois "principais" ao mesmo tempo, o chamado *split-brain*).

1. Casa de volta e estável (conferir saúde por algumas horas).
2. Avisar a manutenção da janela; a EC2 fica em modo leitura ou fora do ar (a API da EC2 é parada).
3. `pg_dump` da EC2 → restaurar no PostgreSQL da casa (substitui o banco antigo); copiar as fotos de volta.
4. Conferir contagens e `GET /v1/health` na casa.
5. Apontar o DNS de volta para a casa; manter a EC2 parada e removida a marca `MODO_RESERVA_ATIVO`.
6. Retomar o backup diário.

9. Metas e custos

9.1 O quanto se perde e quanto demora

Nível	Como	Perda máxima (RPO)	Tempo de troca (RTO)	Esforço
A. Diário (pedido atual)	1 dump por dia às 03:30	até **24 h** de retirada s/devoluções	**~10 a 15 min** (restaura + sobe)	Baixo
B. A cada hora	mesmo script, `OnCalendar=hourly`	até **1 h**	~10 a 15 min	Baixo (banco pequeno: dump leve)
C. Replicação contínua	Replicação lógica/streaming do PostgreSQL pelo túnel	**segundos**	**~3 a 5 min** (banco já está pronto)	Alto (config. de `wal_level`, slots, monitoramento)

Para uma ferramenta que controla **quem está com qual ferramenta**, perder um dia inteiro de movimentação é ruim: a ferramenta aparece "disponível" no sistema restaurado mas está na mão de alguém. O nível **B** custa quase nada e reduz isso a 1 hora; sugiro adotá-lo já na Fase 1. Se a equipe preferir manter "1 por dia" como pedido, documentar essa perda como risco aceito na ata.

9.2 Rateio de custos (us-east-1, sob demanda, cotação US$ 1 = R$ 5,50)

Valores da EC2/EBS/IP/Secrets vêm do guia de deploy (Anexo A); S3, Route 53 e domínio são **estimativas a confirmar** em calculator.aws antes de ir para o relatório.

Componente	Standby frio (EC2 desligada)	Standby quente (EC2 ligada 24 h)
EC2 `t3.medium`	só nas horas de uso (US$ 0,0416/h)	US$ 30,37
EBS gp3 20 GiB	US$ 1,60	US$ 1,60
Elastic IP (IPv4 público)	US$ 3,65	US$ 3,65
Secrets Manager (1 segredo)	US$ 0,40	US$ 0,40
S3 (≈ 2 a 5 GB, 30 dias de retenção)	≈ US$ 0,10	≈ US$ 0,10
Route 53 (zona + 1 health check), opcional	≈ US$ 1,25	≈ US$ 1,25
Domínio `.com.br`, opcional	≈ R$ 40/ano (≈ R$ 3,30/mês)	≈ R$ 3,30/mês
Total AWS (sem Route 53 e domínio)	**≈ US$ 5,75 (≈ R$ 32)**	**≈ US$ 36,12 (≈ R$ 199)**
Rateio por integrante (÷ 4)	**≈ R$ 8**	**≈ R$ 50**

- Uma `t3.small` (US$ 0,0208/h ≈ US$ 15,18/mês; confirmar) basta para este volume e corta o standby quente quase pela metade.
- No Learner Lab o gasto sai do **crédito do lab**, não do bolso; o rateio vale para quando migrar para conta real.
- O Elastic IP é cobrado mesmo com a EC2 parada: não liberar enquanto o domínio apontar para ele.

10. Segurança

- **Nada de segredo no repositório:** senhas, chaves do WireGuard, `JWT_SECRET` e o link de convite ficam no Secrets Manager / `.env` com `chmod 600`.
- PostgreSQL da casa **nunca** exposto à internet: só na interface do túnel, com `backup_ro` somente leitura.
- Bucket S3 **privado**, bloqueio de acesso público ligado, criptografia ligada, versionamento ligado.
- A EC2 reserva só deve ter as portas 22, 80, 443 e UDP 51820. SSH restrito ao IP da equipe.
- Backup contém dados de colaboradores (nome, matrícula): acesso só a quem administra; **não** copiar dump para máquinas pessoais sem necessidade.
- Rotação: se a chave do túnel vazar, gerar par novo nos dois lados; se o `JWT_SECRET` mudar, todos os usuários precisam entrar de novo.

11. Plano de execução

Cada item é um candidato a issue no board (ver `CLAUDE.md`, seção 8; não há numeração atribuída ainda).

Fase 1: backup e restauração comprovados (prioridade, serve para o Dia Maker)

- ☐ Levantar dados do servidor caseiro (seção 12).
- ☐ Criar o bucket S3 privado e a regra de ciclo de vida (30 dias).
- ☐ Montar o túnel WireGuard casa → EC2; testar `ping` e reconexão após queda.
- ☐ Criar `backup_ro` e publicar o PostgreSQL da casa **só** no IP do túnel.
- ☐ Escrever e agendar `backup-pull.sh` (dump + fotos + S3) com `systemd timer`.
- ☐ **Restaurar um dump real na EC2 e comparar contagens.** Registrar evidência (prints/log).
- ☐ Documentar o roteiro manual de troca (seção 7.2) e ensaiar uma vez, cronometrando o RTO.

Fase 2: troca automática

- ☐ Domínio com TTL 60 s e certificado da EC2 por DNS-01.
- ☐ Vigia (`watchdog.sh`) com gatilho duplo (site público + túnel) e alerta.
- ☐ Script de promoção e script de failback.
- ☐ Monitor externo gratuito alertando a equipe.
- ☐ Ensaio completo: desligar a API da casa, medir o tempo até a EC2 responder, voltar.

Fase 3: se sobrar tempo

- ☐ Backup horário (nível B) ou replicação contínua (nível C).
- ☐ Alarme CloudWatch/SNS e painel simples do estado do backup (idade do último dump).
- ☐ Rodar a API da reserva em modo somente leitura enquanto a casa estiver viva (consulta rápida).

☐ Incluir o backup diário no relatório de otimização/monitoramento (entrega de 03/11) e no de segurança.

12. Pendências: dados que ninguém tem ainda

#	O que falta saber	Por que importa
1	Sistema operacional e hardware do servidor caseiro; versão do Docker e do Dokploy	Instalar WireGuard e definir volumes
2	Versão do PostgreSQL da imagem do Dokploy	`pg_dump` da EC2 precisa ser igual ou mais novo
3	A casa tem IP público fixo, dinâmico ou está atrás de CGNAT?	Define se o túnel só pode ser iniciado pela casa (o desenho já assume isso)
4	Domínio definitivo (próprio ou DuckDNS)	Sem isso não há failover por DNS
5	O Learner Lab libera Route 53, SNS, EventBridge/Lambda?	Define o que vira script na EC2
6	Caminho real dos volumes de fotos no host	Backup das fotos
7	Decisão da equipe: RPO 24 h aceito, ou backup horário?	Seção 9.1
8	A EC2 de reserva fica "quente" (US$ 36) ou "fria" (US$ 5,75, só em conta real)?	Seção 9.2
9	Quem é o responsável e quem é o reserva por esta peça?	Sugestão: Guilherme (infra), com Henrique no banco
10	Falta uma ata registrando a decisão (regra do projeto: reunião de quarta, ata em `/docs/atas/`)	Rastreabilidade para o PI

Apêndice A: esboços de script (a validar na EC2, não testados)

`backup-pull.sh` *(EC2,* `systemd timer` *diário)*

```
#!/usr/bin/env bash
set -euo pipefail

CASA_WG_IP="10.8.0.2"
DATA="$(date +%F_%H%M)"
DESTINO="/var/backups/soufer"
BUCKET="s3://soufer-tools-backup-SUFIXO"
REGIAO="us-east-1"

mkdir -p "$DESTINO/fotos"

PGPASSWORD="$(aws secretsmanager get-secret-value --region "$REGIAO" \
  --secret-id soufer-tools/backup-ro --query SecretString --output text | jq -r .password)"
export PGPASSWORD

# 1) banco
pg_dump -Fc -h "$CASA_WG_IP" -U backup_ro soufer_prod -f "$DESTINO/soufer_${DATA}.dump"
pg_restore -l "$DESTINO/soufer_${DATA}.dump" > /dev/null        # valida o arquivo

# 2) fotos (ajustar o caminho do volume no host da casa)
rsync -a -e "ssh -i /home/ubuntu/.ssh/backup_casa" \
  "backup@${CASA_WG_IP}:/CAMINHO/DO/VOLUME/" "$DESTINO/fotos/"

# 3) nuvem
aws s3 cp "$DESTINO/soufer_${DATA}.dump" "$BUCKET/$(date +%F)/" --sse AES256
aws s3 sync "$DESTINO/fotos/" "$BUCKET/fotos/" --sse AES256

# 4) limpeza local (o S3 tem regra de ciclo de vida)
find "$DESTINO" -maxdepth 1 -name '*.dump' -mtime +14 -delete
```

`watchdog.sh` *(EC2, a cada 30 s): lógica*

```
falhas = ler contador em /var/lib/soufer/falhas
se existe MODO_RESERVA_ATIVO: sair (reserva já é a principal; failback é manual)
se GET https://app.<dominio>/v1/health devolve status "ok" em 5 s: falhas = 0; sair
senão: falhas += 1
se falhas >= 6 e último handshake do WireGuard tem mais de 3 min:
    alertar "casa fora do ar"; executar promover.sh
senão se falhas >= 6: alertar "site fora, túnel vivo: investigar" (sem trocar)
```